

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.rcParams["figure.autolayout"] = True # to for tight_layout()
import pymc as pm
import xarray as xr
import scipy as sp
from scipy.special import expit as logistic
import pytensor.tensor as tt
```

Exam group formation in LearnIT

Remember to create/join a group for the exam in LearnIT. More info [here](#).

Mixtures (Generalized GLMs)

- More realistic, try to model reality (instead of transforming data to fit the model)

Agenda

- Over-dispersion
 - Beta-Binomial
 - Gamma-Poisson
- Zero inflated outcomes
 - Zero-Inflated Poisson
- Ordinal Regression
 - Ordinal outcome (predicted variable)
 - Ordinal predictor



Over-dispersion

- Remember that Gaussian models were very sensitive to outliers
 - The inferred variance of the distributions was large
 - **[QUESTION]: What could we do to alleviate this issue?**
- Count models also may also be sensitive to outliers
- When data is more variable than what a pure model assumes, then we say that it exhibits **over-dispersion**
- Often over-dispersion appears due to *missing predictors*
 - Ignoring over-dispersion may lead to spurious inferences
 - The mixture models we will discuss may help to capture over-dispersion
 - However, when possible, it is better to include any missing predictors causing the over-dispersion

Beta-Binomial

Beta distribution (revisited)

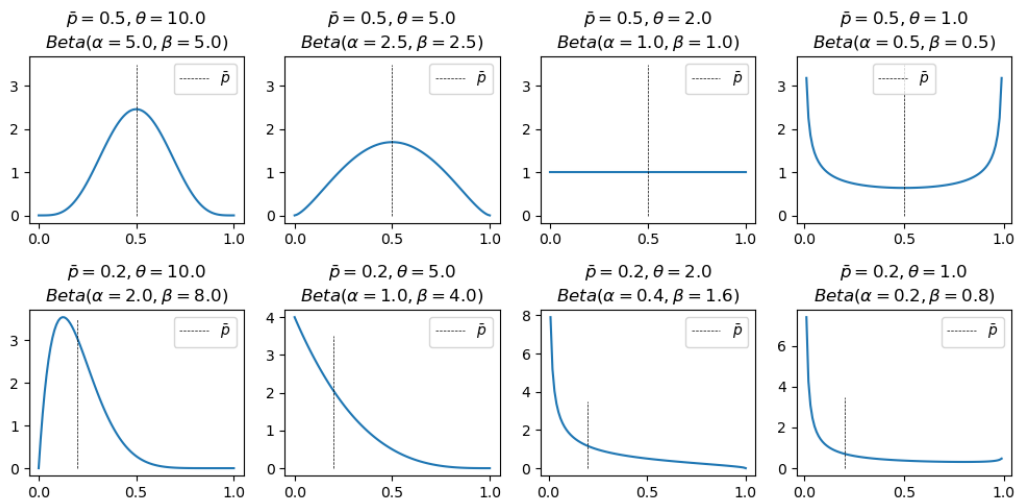
- The *Beta* distribution is a continuous distribution whose support is (0,1)
- The Beta distribution has two shape parameters α and β , $\text{Beta}(\alpha, \beta)$

$$p(\theta | \alpha, \beta) = \frac{\theta^{\alpha-1}(1-\theta)^{\beta-1}}{B(\alpha, \beta)}$$

where $B(\alpha, \beta) = \int_0^1 \theta^{\alpha-1}(1-\theta)^{\beta-1} d\theta$ is the normalizing factor

- These parameters define the shape of the distribution
- The parameters α and β do not have a very intuitive interpretation
- The book uses an alternative way to define the shape of the distribution in terms of:
 - The average probability ($\bar{p}$) and,
 - shape (θ) or concentration (when $\theta = 2$ means equal probabilities, $\theta > 2$ greater concentration around $\bar{p}$ and $\theta < 2$ more dispersion)
- Given the above parameters, we can define $\text{Beta}(\alpha = \bar{p}\theta, \beta = (1 - \bar{p})\theta)$

```
In [2]: x_plot = np.linspace(0, 1, 100)
pbars = [0.5, 0.5, 0.5, 0.5, 0.5, 0.2, 0.2, 0.2, 0.2]
thetas = [10.0, 5.0, 2.0, 1.0, 10.0, 5.0, 2.0, 1.0]
fig, axs = plt.subplots(2, 4, figsize=(10,5))
for pbar, theta, ax in zip(pbars, thetas, axs.ravel()):
    ax.plot(
        x_plot,
        sp.stats.beta.pdf(x_plot[:, np.newaxis], pbar * theta, (1 - pbar) * theta)
    )
    ax.vlines(x=pbar, ymin=0, ymax=3.5, linestyle='--', color='black', lw=.5, label=f'$\bar{p}$')
    ax.set_title(label=f'$\bar{p}$ = {pbar}, \theta = {theta}$\nBeta($\alpha$={pbar * theta}, $\beta$={(1 - pbar) * theta})$')
    ax.legend();
```



Beta-Binomial Model

- Key: It is a **mixture** of binomial distributions
- This model assumes a different p for each row in the dataset
 - That is, each row has its own unobserved success rate
 - We do not estimate a distribution of probabilities of success, but a **distribution of distributions of probability of success**
- The model makes use of the conjugacy property of Beta priors and Binomial likelihoods
- Recall the UC admissions model, data was dispersed if we ignore the predictor "department"

```
In [3]: d_ucb = pd.read_csv("Data/UCBadmit.csv", sep=";")
N = d_ucb.applications.values
gid = (d_ucb["applicant.gender"] == "female").astype(int).values
d_ucb.head()
```

```
Out[3]:
```

	dept	applicant.gender	admit	reject	applications
1	A	male	512	313	825
2	A	female	89	19	108
3	B	male	353	207	560
4	B	female	17	8	25
5	C	male	120	205	325

- We apply a Beta-Binomial model with a categorical predictor (gid)

$$A_i \sim \text{BetaBinomial}(N_i, \bar{p}_i, \theta)$$

$$\text{logit}(\bar{p}_i) = \alpha_{\text{GID}[i]}$$

$$\alpha_j \sim \text{Normal}(0, 1.5)$$

$$\theta = \varphi + 2$$

$$\varphi = \text{Exponential}(1)$$

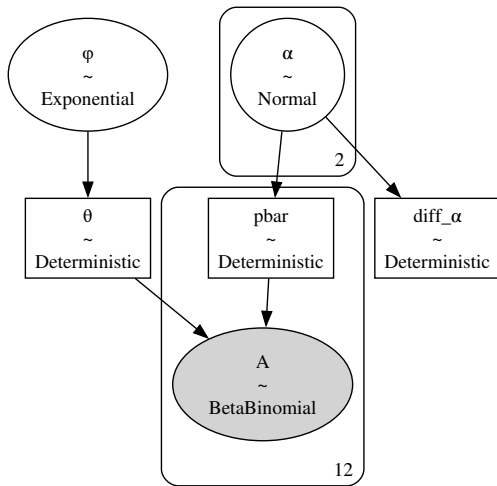
- [QUESTION]: Why do we define $\theta = \varphi + 2$?

```
In [4]: with pm.Model() as m12_1:
    phi = pm.Exponential('phi', lam=1)
    theta = pm.Deterministic('theta', phi + 2) # so that it is part of the trace
    alpha = pm.Normal('alpha', mu=0, sigma=1.5, shape=np.unique(gid).size)
    pbar = pm.Deterministic('pbar', pm.invlogit(alpha[gid])) # so that it is part of the trace
    A = pm.BetaBinomial('A', n=N, alpha=pbar*theta, beta=(1-pbar)*theta, observed=d_uch.admit.values)

    # difference of log-odds and probability scale
    pm.Deterministic('diff_alpha', alpha[0]-alpha[1])
```

```
In [5]: pm.model_to_graphviz(m12_1)
```

```
Out[5]:
```



```
In [6]: trace_m12_1 = pm.sample(model=m12_1, return_inferencedata=True)
```

Initializing NUTS using jitter+adapt_diag...
 Multiprocess sampling (4 chains in 4 jobs)
 NUTS: [phi, alpha]
 Output()
 Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 1 seconds.

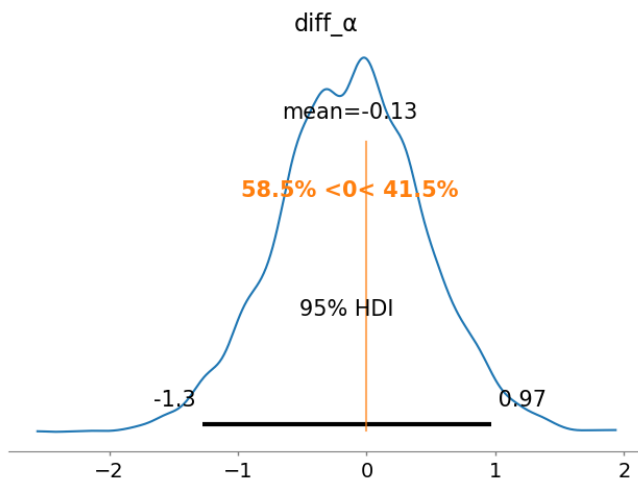
```
In [7]: pm.summary(trace_m12_1, var_names=['alpha', 'phi', 'theta', 'diff_alpha'])
```

```
Out[7]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
alpha[0]	-0.442	0.410	-1.252	0.298	0.008	0.006	3013.0	2283.0	1.0
alpha[1]	-0.314	0.413	-1.094	0.478	0.007	0.006	3143.0	2812.0	1.0
phi	1.036	0.800	0.000	2.496	0.015	0.011	1971.0	1467.0	1.0
theta	3.036	0.800	2.000	4.496	0.015	0.011	1971.0	1467.0	1.0
diff_alpha	-0.128	0.574	-1.261	0.910	0.010	0.009	3152.0	2641.0	1.0

- [Question]: In what scale are the α parameters?
- Note that the contrast diff_alpha is negative, but also quite widespread. Meaning that there is not evidence for a bias in the admission rates
 - The following posterior plot visualizes this

```
In [8]: pm.plot_posterior(trace_m12_1, var_names=['diff_alpha'], ref_val=0.0, hdi_prob=0.95);
```



- It is important to note that, in this Beta-Binomial model, the $\bar{p}$ and θ samples give us a **distribution of Beta posterior distributions**
 - Note the difference with the regular Binomial regression model that gives a distribution over values of p
 - [Question]: Why do we know that the posterior samples of $\bar{p}$ and θ define Beta distributions?**
- The plot below shows that there is a lot of variation
 - Overall all $\bar{p}$ values are plausible

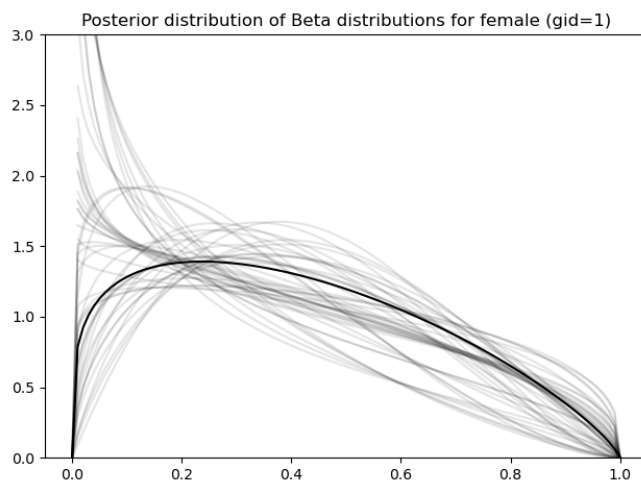
```
In [9]: gid = 1
x_plot = np.linspace(0, 1, 100)

trace_pbar_gid = trace_m12_1.posterior.sel(α_dim_θ=gid)['pbar'].values.ravel()
trace_θ_gid = trace_m12_1.posterior.sel(α_dim_θ=gid)['θ'].values.ravel()

trace_pbar_gid_mean = trace_pbar_gid.mean()
trace_θ_gid_mean = trace_θ_gid.mean()

# mean Beta distribution
plt.plot(x_plot, sp.stats.beta.pdf(x_plot, trace_pbar_gid_mean*trace_θ_gid_mean,
                                   (1-trace_pbar_gid_mean)*trace_θ_gid_mean), color='k');

# 50 sampled Beta posterior distributions
n_samples = 50
for pbar, theta in zip(trace_pbar_gid[-n_samples:], trace_θ_gid[-n_samples:]):
    plt.plot(x_plot, sp.stats.beta.pdf(x_plot, pbar*theta, (1-pbar)*theta), alpha=.1, color='k');
plt.title(f'Posterior distribution of Beta distributions for female (gid={gid})');
plt.ylim(0.0, 3.0)
plt.show();
```



- Finally, we perform a posterior predictive check to assess whether the model fits the data
- Although it does, we observe that the model produces notably predictions

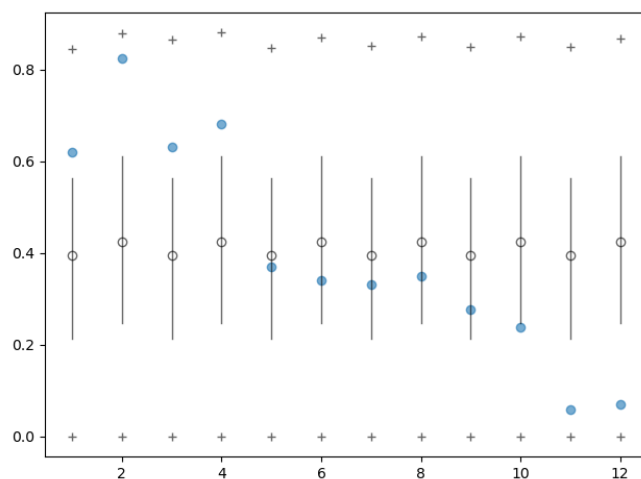
```
In [10]: with m12_1:
pcc_test = pm.sample_posterior_predictive(trace_m12_1)

Sampling: [A]
Output()

In [11]: with m12_1:
pcc = pm.sample_posterior_predictive(trace_m12_1).posterior_predictive["A"]
pp_admit = pcc / N

Sampling: [A]
Output()

In [12]: # adapted from https://github.com/pymc-devs/pymc-resources/blob/main/Rethinking_2/Chp_12.ipynb
trace_m12_1_pbar = np.concatenate(trace_m12_1.posterior["pbar"], axis=0)
plt.plot(range(1, 13), d_uch.admit / N, "C0o", ms=6, alpha=0.6)
plt.plot(range(1, 13), trace_m12_1_pbar.mean(0), "ko", fillstyle="none", ms=6, alpha=0.6)
plt.plot([range(1, 13), range(1, 13)], pm.hdi(trace_m12_1_pbar[None, :]).T, "k-", lw=1, alpha=0.6)
plt.plot([range(1, 13), range(1, 13)], pm.hdi(pp_admit)['A'].T, "k+", ms=6, alpha=0.6);
```



Gamma-Poisson

- [QUESTION]: How do Binomial and Poisson models differ?
- The Gamma distribution is a generalization of the Exponential distribution
 - The Gamma distribution is a conjugate prior for Poisson models
 - As in Beta-Binomial, the Gamma distribution is chosen due to mathematical convenience
- The Gamma-Poisson model assumes a different rate parameter (λ) for each row in the dataset.
- The Gamma-Poisson distribution has two parameters, rate (λ) and dispersion or scale (φ)

$$\text{Gamma-Poisson}(\lambda, \varphi)$$

Gamma-Poisson Model

- We revisit the oceanic islands and tools (scientific) model
- Now we use a Gamma-Poisson model to account for over-dispersion
- The model is defined as

$$\begin{aligned} T_i &\sim \text{Gamma-Poisson}(\lambda_i, \varphi) \\ \lambda_i &= \frac{\alpha_{\text{CID}[i]} P_i^{\beta_{\text{CID}[i]}}}{\gamma} \\ \alpha_i &\sim \text{Normal}(1, 1) \\ \beta_i &\sim \text{Exponential}(1) \\ \gamma &\sim \text{Exponential}(1) \\ \varphi &\sim \text{Exponential}(1) \end{aligned}$$

```
In [13]: d_k = pd.read_csv("Data/Kline.csv", delimiter=";")
c_id = (d_k.contact == "high").astype(int).values
d_k
```

```
Out[13]:
```

	culture	population	contact	total_tools	mean_TU
0	Malekula	1100	low	13	3.2
1	Tikopia	1500	low	22	4.7
2	Santa Cruz	3600	low	24	4.0
3	Yap	4791	high	43	5.0
4	Lau Fiji	7400	high	33	5.0
5	Trobriand	8000	high	19	4.0
6	Chuuk	9200	high	40	3.8
7	Manus	13000	low	28	6.6
8	Tonga	17500	high	55	5.4
9	Hawaii	275000	low	71	6.6

```
In [14]: # from previous lecture
with pm.Model() as m11_11:
    y = pm.Exponential('y',1)
    beta = pm.Exponential('beta',1,shape=np.unique(c_id).size)
    alpha = pm.LogitNormal('alpha',1,1,shape=np.unique(c_id).size)
    cid = pm.Data("cid", c_id)
    P = pm.Data("P", d_k.population)
    lambda = pm.Deterministic('lambda', alpha[cid]*tt.math.pow(P,beta[cid])/y)
    T = pm.Poisson('T',mu=lambda,observed=d_k.total_tools)
    trace_m11_11 = pm.sample(idata_kwargs={'log_likelihood': True})

# Gamma-Poisson models
with pm.Model() as m12_2:
    y = pm.Exponential('y',1)
    beta = pm.Exponential('beta',1,shape=np.unique(c_id).size)
    alpha = pm.LogitNormal('alpha',1,1,shape=np.unique(c_id).size)
    cid = pm.Data("cid", c_id)
    P = pm.Data("P", d_k.population)
    lambda = pm.Deterministic('lambda', alpha[cid]*tt.math.pow(P,beta[cid])/y)

    # only new changes to turn it in to a Gamma-Poisson
    phi = pm.Exponential('phi',1)
    T = pm.NegativeBinomial('T',lambda,phi,observed=d_k.total_tools)
    trace_m12_2 = pm.sample(idata_kwargs={'log_likelihood': True})
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [y, beta, alpha]
Output()
```

```
Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 3 seconds.
There were 6 divergences after tuning. Increase `target_accept` or reparameterize.
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [y, beta, alpha, phi]
Output()
```

```
Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 4 seconds.
There were 2 divergences after tuning. Increase `target_accept` or reparameterize.
```

```
In [15]: # adapted from https://github.com/pymc-devs/pymc-resources/blob/main/Rethinking_2/Chp_12.ipynb
ns = 10
P_seq = d_k.population

with m11_11:
    # predictions for cid=0 (low contact)
    pm.set_data({"cid": np.array([0] * ns), "P": P_seq})
    lam0_11 = pm.sample_posterior_predictive(trace_m11_11, var_names=["T"])[
        "posterior_predictive"
    ][0]

    # predictions for cid=1 (high contact)
    pm.set_data({"cid": np.array([1] * ns)})
    lam1_11 = pm.sample_posterior_predictive(trace_m11_11, var_names=["T"])[
        "posterior_predictive"
    ][0]
```

```

    ]["T"]

with m12_2:
    pm.set_data({"cid": np.array([0] * ns), "P": P_seq})
    lam0_12 = pm.sample_posterior_predictive(trace_m12_2, var_names=["T"])[
        "posterior_predictive"
    ]["T"]

    pm.set_data({"cid": np.array([1] * ns)})
    lam1_12 = pm.sample_posterior_predictive(trace_m12_2, var_names=["T"])[
        "posterior_predictive"
    ]["T"]

lmu0_11, lmu1_11 = lam0_11.mean(["chain", "draw"]), lam1_11.mean(["chain", "draw"])
lmu0_12, lmu1_12 = lam0_12.mean(["chain", "draw"]), lam1_12.mean(["chain", "draw"])

```

Sampling: [T]
Output()

Sampling: [T]
Output()

Sampling: [T]
Output()

Sampling: [T]
Output()

```

In [ ]: # scale point size to Pareto-k:
        k_11 = pm.loo(trace_m11_11, pointwise=True).pareto_k.values

```

```

In [17]: k_12 = pm.loo(trace_m12_2, pointwise=True).pareto_k.values

```

- Note no Pareto distribution warning in the Gamma-Poisson model
- The posterior predictive check below shows a much higher level of variation in the Gamma-Poisson model (as expected)

```

In [18]: # adapted from https://github.com/pymc-devs/pymc-resources/blob/main/Rethinking_2/Chp_12.ipynb

k_11 /= k_11.max()
psize_11 = 250 * k_11
k_12 /= k_12.max()
psize_12 = 250 * k_12

_, (ax0, ax1) = plt.subplots(1, 2, figsize=(10, 6))

# Pure poisson predictions:
pm.plot_hdi(P_seq, lam1_11, color="b", fill_kwargs={"alpha": 0.2}, ax=ax0)
ax0.plot(P_seq, lmu1_11, color="b", alpha=0.7, label="high contact mean")

pm.plot_hdi(P_seq, lam0_11, color="k", fill_kwargs={"alpha": 0.2}, ax=ax0)
ax0.plot(P_seq, lmu0_11, "--", color="k", alpha=0.7, label="low contact mean")

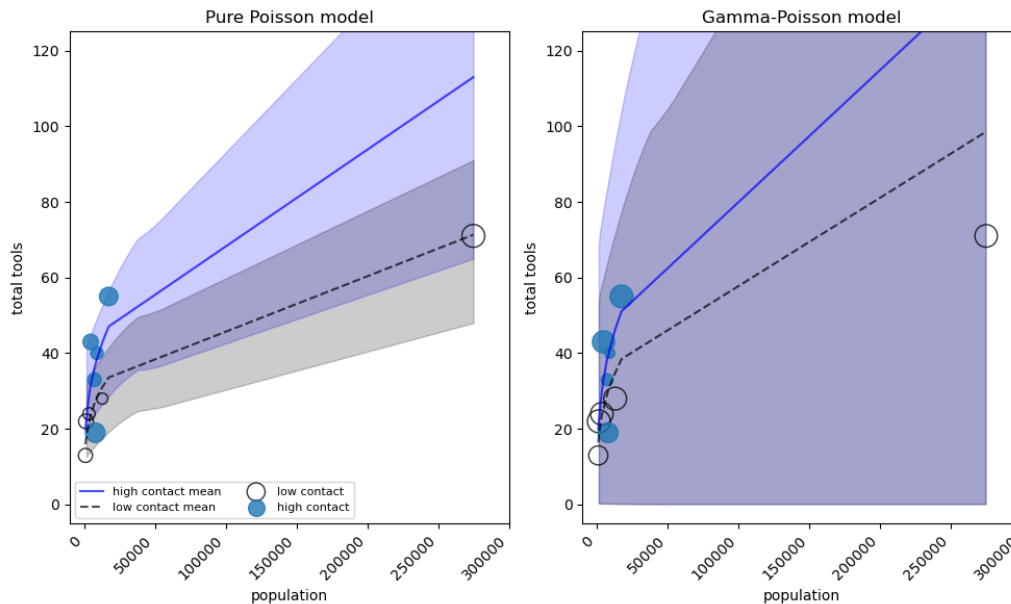
# display observed data:
index = c_id == 1
ax0.scatter(
    d_k.population[~index], d_k.total_tools[~index], s=psize_11[~index],
    facecolors="none", edgecolors="k", alpha=0.8, lw=1, label="low contact",
)
ax0.scatter(
    d_k.population[index], d_k.total_tools[index], s=psize_11[index], alpha=0.8, label="high contact"
)
plt.setp(ax0.get_xticklabels(), ha="right", rotation=45)
ax0.set_xlim((-10_000, 300_000))
ax0.set_xlabel("population")
ax0.set_ylabel("total tools")
ax0.set_ylim((-5, 125))
ax0.set_title("Pure Poisson model")
ax0.legend(fontsize=8, ncol=2)

# Gamma-Poisson predictions:
pm.plot_hdi(P_seq, lam1_12, color="b", fill_kwargs={"alpha": 0.2}, ax=ax1)
ax1.plot(P_seq, lmu1_12, color="b", alpha=0.7)

pm.plot_hdi(P_seq, lam0_12, color="k", fill_kwargs={"alpha": 0.2}, ax=ax1)
ax1.plot(P_seq, lmu0_12, "--", color="k", alpha=0.7)

# display observed data:
ax1.scatter(
    d_k.population[~index], d_k.total_tools[~index], s=psize_12[~index],
    facecolors="none", edgecolors="k", alpha=0.8, lw=1
)
ax1.scatter(
    d_k.population[index], d_k.total_tools[index], s=psize_12[index], alpha=0.8
)
plt.setp(ax1.get_xticklabels(), ha="right", rotation=45)
ax1.set_xlim((-10_000, 300_000))
ax1.set_xlabel("population")
ax1.set_ylabel("total tools")
ax1.set_ylim((-5, 125))
ax1.set_title("Gamma-Poisson model");

```



Zero inflated outcomes

- Recorded data may not come from pure models
- Often, data come from multiple processes
- As we have seen above, mixture models allow us to capture several processes in one model
- In counting models, zero is particularly important
 - It might result from the processes not occurring
 - The process not progressing enough to produce anything
 - These different sources of zero outcomes produce **zero inflated models**

Zero-Inflated Poisson: Writing Monastery

- We look at a zero-inflated Poisson model
- Consider a monastery where monks produce a number of manuscripts every day.
- We would like to predict the daily rate at which monks produce manuscripts
 - In order words, how many manuscripts monks produce per day on average
- Monks do not work every day, some days they take breaks for wine tasting, those days they produce zero manuscripts
- However, it is also possible that monks produce zero manuscripts because, that day, the writing process went slowly.
- The process is illustrated in the figure below



<https://en.wikipedia.org/wiki/File:Escribano.jpg>

Image source: Wikipedia

- We use a zero-inflated Poisson model to estimate: i) the probability of the monks going to the wine cellar and ii) the rate of manuscripts they produce per day
 - We use p to denote the probability that the monks go to the wine cellar
 - We use λ to denote the rate of manuscript production

Likelihood for zero-inflated model

- To better understand the likelihood of this model, let's think of the data generating process as a mix of two processes:
 - With probability p we select a process that always produces 0 manuscripts
 - With probability $1 - p$ we select a process that generates manuscripts with rate λ
- Given the above, the likelihood of producing a zero manuscript is given by

$$\begin{aligned} \Pr(0 \mid p, \lambda) &= \Pr(\text{drink} \mid p) \cdot \underbrace{1}_{\text{always no manuscripts when drinking}} + \Pr(\text{work} \mid p) \cdot \Pr(0 \mid \lambda) \\ &= p + (1 - p) \cdot \exp(-\lambda) \end{aligned}$$

where $\exp(-\lambda)$ is the Poisson pmf for outcome 0

- Similarly, the likelihood of producing non-zero manuscripts is given by

$$\begin{aligned}\Pr(y > 0 \mid p, \lambda) &= \Pr(\text{drink} \mid p) \cdot \underbrace{0}_{\text{no manuscripts when drinking}} + \Pr(\text{work} \mid p) \cdot \Pr(y > 0 \mid \lambda) \\ &= 0 + (1 - p) \cdot \underbrace{\frac{\lambda^y \exp(-\lambda)}{y!}}_{\text{Poisson pmf}}\end{aligned}$$

- This distribution exists and it is known as ZIPoisson.

Data generating process

- We build a generative model for the data, and try to recover the value of the parameters using the ZIPoisson model

```
In [19]: # define parameters
PROB_DRINK = 0.2 # 20% of days
RATE_WORK = 1.0 # average 1 manuscript per day

# sample one year of production
N = 365

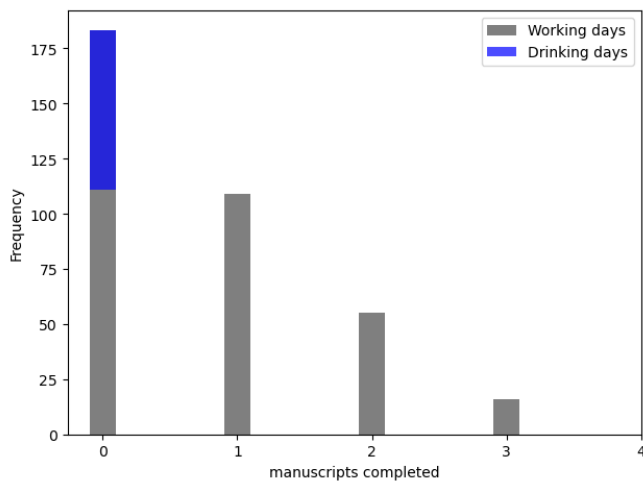
# simulate days monks drink
drink = np.random.binomial(1,PROB_DRINK, size=N)

# simulate manuscripts completed
y = (1 - drink) * np.random.poisson(RATE_WORK, size=N)

In [20]: zeros_drink = drink.sum()
zeros_work = (y == 0).sum() - zeros_drink

bins = np.arange(y.max() + 1) - 0.5
plt.hist(y, bins=bins, align="mid", rwidth=0.2, color="k", alpha=0.5, label="Working days")
plt.bar(0.0, zeros_drink, bottom=zeros_work, width=0.2, color="b", alpha=0.7, label="Drinking days")

plt.xticks(bins + 0.5)
plt.xlabel("manuscripts completed")
plt.legend()
plt.ylabel("Frequency");
```



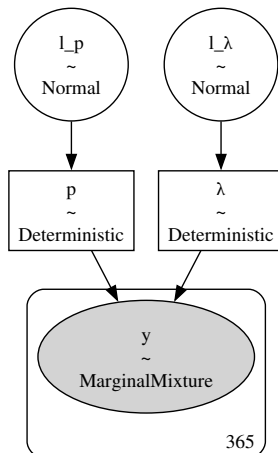
Zero-Inflated Poisson model

- Note that in the PyMC function `ZeroInflatedPoisson` we specify the probability of getting a 0
 - <https://www.pymc.io/projects/docs/en/latest/api/distributions/generated/pymc.ZeroInflatedPoisson.html#pymc.ZeroInflatedPoisson>
 - In our case study, this means the probability that the monks are on a working day (i.e., $1 - p$)

```
In [21]: with pm.Model() as m12_3:
l_λ = pm.Normal('l_λ', mu=1, sigma=.5)
l_p = pm.Normal('l_p', mu=-1.5, sigma=1)
λ = pm.Deterministic('λ', pm.math.exp(l_λ))
p = pm.Deterministic('p', pm.math.invlogit(l_p))
y = pm.ZeroInflatedPoisson('y', 1-p, λ, observed=y)
```

```
In [22]: pm.model_to_graphviz(m12_3)
```

```
Out[22]:
```



```
In [23]: trace_m12_3 = pm.sample(model=m12_3)
```



```

Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [l_λ, l_p]
Output()

Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 2 seconds.

```

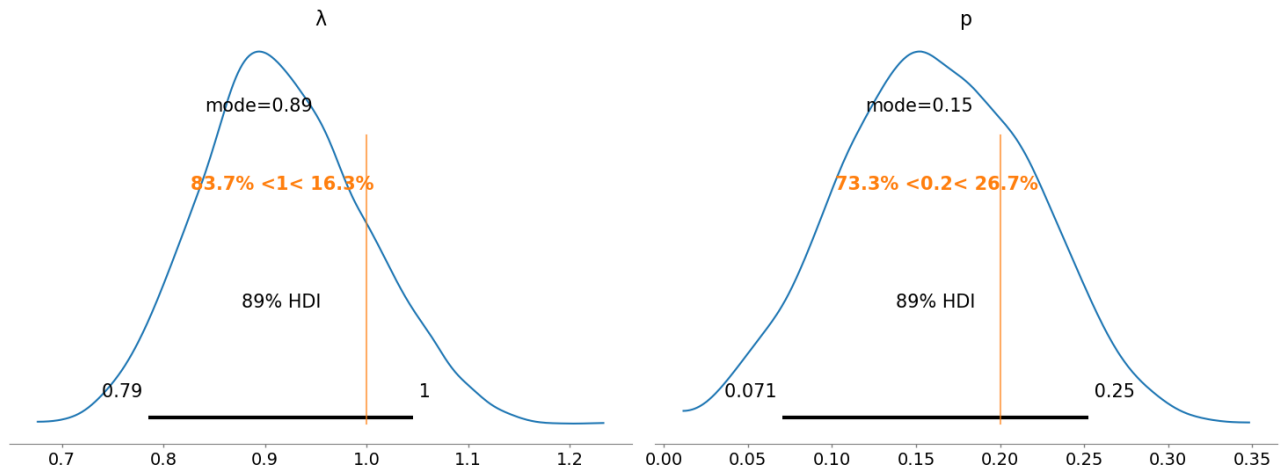
```
In [24]: pm.summary(trace_m12_3)
```

```

Out[24]:
      mean      sd  hdi_3%  hdi_97%  mcse_mean  mcse_sd  ess_bulk  ess_tail  r_hat
l_λ -0.090  0.088   -0.259   0.071     0.002    0.002   1273.0   1436.0    1.0
l_p -1.715  0.485   -2.618  -0.896     0.014    0.010   1285.0   1336.0    1.0
  λ   0.917  0.081    0.768   1.069     0.002    0.002   1273.0   1436.0    1.0
  p   0.162  0.057    0.053   0.263     0.002    0.001   1285.0   1336.0    1.0

```

```
In [25]: pm.plot_posterior(trace_m12_3, var_names=['λ', 'p'], hdi_prob=.89, point_estimate='mode', ref_val=[RATE_WORK, PROB_DRINK]);
```



- The posterior of the parameters is very close to the original values
 - $\lambda \approx 1 = \text{RATE_WORK}$
 - $p \approx 0.2 = \text{PROB_DRINK}$

Ordered Categorical Regression (Ordinal regression)

- Sometimes data is represented as a *discrete* indicative value
 - Positions in a race
 - Answers to questions in course evaluation surveys (*e.g.*, how much you agree with a statement)
 - This type of data is represented in an *ordinal scale*
- The models we have considered so far worked on a *metric* scale
 - (except for multinomial regression, whose data is expressed in a *nominal* scale)
- The main difference with metric scales is that there is notion of distance between elements
 - For instance, in the manuscript writing model, we can tell difference of manuscripts produced in two different days
- Consider the example of the positions in a race
 - In an ordered scale, we can tell who finished first, second and so on
 - But we do not require that the distance between the first and the second is the same as second and third, or any other pair of positions.
 - It is possible that the difference between 1 and 2 was 1 second, and between 2 and 3 5 seconds.

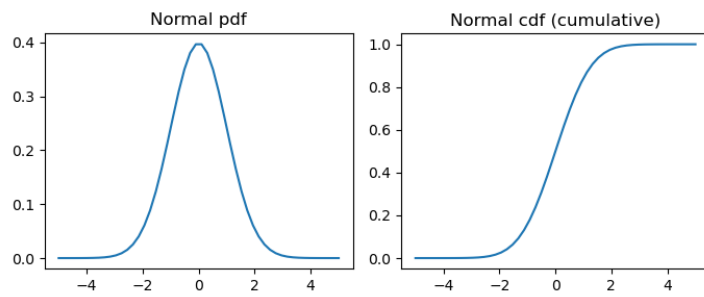
Ordered categorical outcomes

- First we consider the case where the outcome variable is in ordinal scale
- In a nutshell, we re-use the standard multinomial model with the constraint that outcomes follow some order
 - The model should ensure that, if a predictor is positively related to the ordered outcome, then larger values of the predictor increase the value in the ordinal scale
 - For instance, if age of the respondent is positively related to high values in course evaluations, the model must ensure that predictions increase as age increases
- The standard solution to the previous point is using a **cumulative link function**.
 - Typically these are *cumulative probability/density functions*.
 - For instance, for a discrete distribution with positive support X , the cumulative probability of outcome 3 is $Pr(X \leq 3) = Pr(X = 1) + Pr(X = 2) + Pr(X = 3)$
- More concretely, we will use a *cumulative-log-odds*

```

In [26]: X = np.linspace(-5,5,50)
fig, (ax1, ax2) = plt.subplots(1,2,figsize=(7,3))
ax1.plot(X, sp.stats.norm.pdf(X)); ax1.set_title('Normal pdf')
ax2.plot(X, sp.stats.norm.cdf(X)); ax2.set_title('Normal cdf (cumulative)');

```



Ethical Trolley

 https://en.wikipedia.org/wiki/Trolley_problem#/media/File:Trolley_Problem



Image source: Wikipedia

- We consider a dataset where respondents rate: How morally permissible is to pull the lever?
 - We use an ordinal scale from 1 (least permissible) to 7 (most permissible)

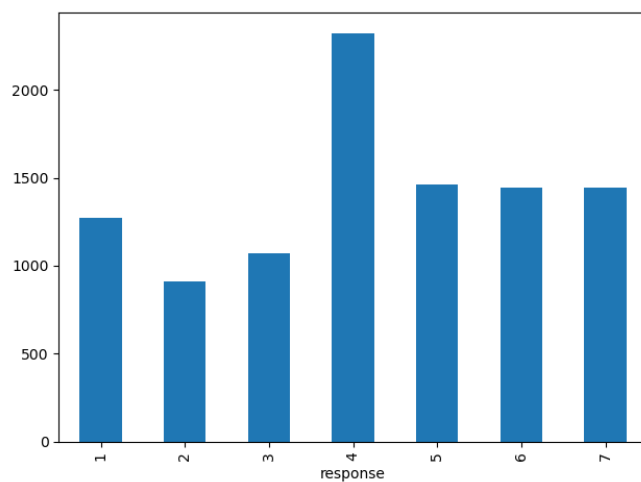
Data

```
In [27]: d_tr = pd.read_csv("Data/Trolley.csv", sep=";")
d_tr.head()
```

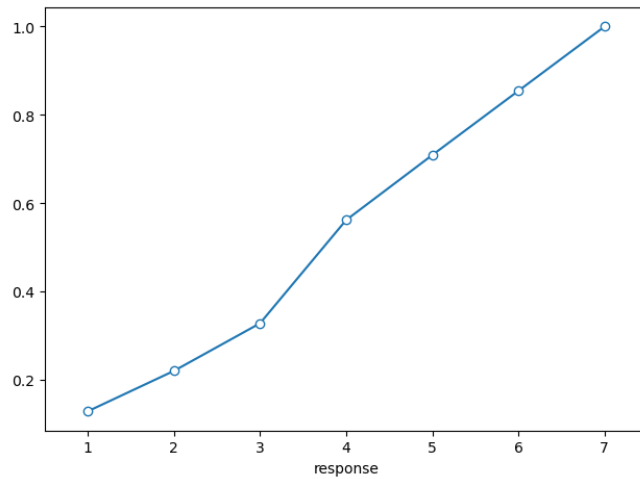
```
Out[27]:
```

	case	response	order	id	age	male	edu	action	intention	contact	story	action2
0	cfaqu	4	2	96;434	14	0	Middle School	0	0	1	aqu	1
1	cfbur	3	31	96;434	14	0	Middle School	0	0	1	bur	1
2	cfруб	4	16	96;434	14	0	Middle School	0	0	1	руб	1
3	cibox	3	32	96;434	14	0	Middle School	0	1	1	box	1
4	cibur	3	4	96;434	14	0	Middle School	0	1	1	bur	1

```
In [28]: d_tr.response.value_counts(sort=False).sort_index().plot(kind='bar');
```



```
In [29]: pr=d_tr.response.value_counts(normalize=True,sort=False).sort_index().cumsum()
pr.plot(marker='o',mfc='w')
plt.xlim(0.5, 7.5);
```



Cumulative-log-odds

- Turning the cumulative probabilities above into cumulative-log-odds defines an intercept for each outcome as expected

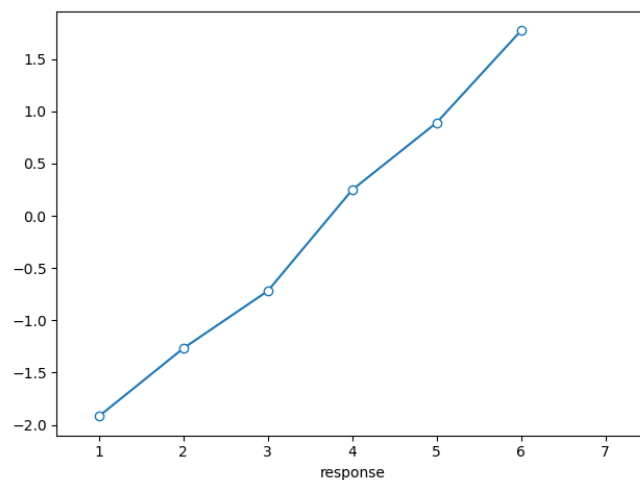
$$\log \frac{\Pr(y_i \leq k)}{1 - \Pr(y_i \leq k)} = \alpha_k$$

- The concrete value for each intercept in the data is printed and plotted below

```
In [30]: d_tr.response.value_counts(normalize=True, sort=False).sort_index().cumsum().apply(sp.special.logit).values.round(2)
```

```
Out[30]: array([-1.92, -1.27, -0.72,  0.25,  0.89,  1.77,  nan])
```

```
In [31]: d_tr.response.value_counts(normalize=True, sort=False).sort_index().cumsum().apply(sp.special.logit).plot(marker='o', mfc='w')
plt.xlim(0.5, 7.5);
```



- Note that $\alpha_7 = \infty$
 - [QUESTION]: why?**
- In fact, only need to know the probability of all outcomes except 1 ($K - 1$)
 - The law of total probability states that $\sum_i \Pr(X = i) = 1$
 - Knowing the probability of all outcomes except for one tells us the missing probability of the outcome

Probability for each outcome

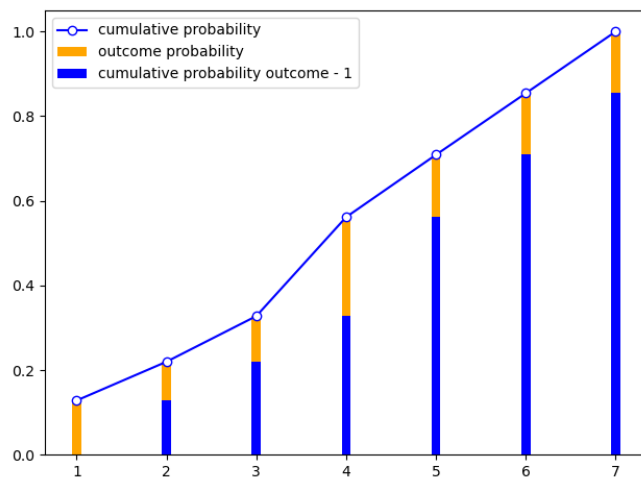
- So far we have discussed only cumulative probability, but we are actually interested in the probability for each outcome (not the cumulative)
- It is easy to use cumulative probabilities to compute the probability of each outcome
- Let p_k denote the probability of outcome k

$$p_k = \Pr(y_i \leq k) - \Pr(y_i \leq k - 1)$$

That is, the probability of an outcome k is the cumulative probability for k minus the cumulative probability of $k - 1$

- Now we show this graphically
 - Another version of the figure 12.5 in the textbook
 - We set the probability of outcomes as a bar from the bottom of the y-axis

```
In [32]: X = np.arange(1,8)
plt.plot(X,pr,marker='o',mfc='w', label='cumulative probability', color='blue')
plt.bar(X,pr,width=.1, label='outcome probability', color='orange')
outcome_pr = [0,
               pr[1],
               pr[2],
               pr[3],
               pr[4],
               pr[5],
               pr[6]]
plt.bar(X,outcome_pr,width=.1, label='cumulative probability outcome - 1', color='blue')
plt.legend();
```



Ordered Regression for ethical trolley

- We consider the following model

$$\begin{aligned}
 R_i &\sim \text{Categorical}(\vec{p}) \\
 p_1 &= q_1 \\
 p_k &= q_k - q_{k-1} \text{ where } k \in \{2, \dots, K-2\} \\
 p_K &= 1 - q_{K-1} \\
 \text{logit}(q_k) &= \kappa_k - \varphi_i \\
 \varphi_i &= \text{terms of the linear model} \\
 \kappa_k &\sim \text{Normal}(0, 1.5)
 \end{aligned}$$

where q_i is the cumulative-log-odds for each outcome, p_i the probability of each outcome, and $\vec{p} = \langle p_1, p_2, \dots, p_K \rangle$.

- [QUESTION]:** Is $\text{Normal}(0, 1.5)$ a good prior for κ_k ? Why?
- This model is so common that there exists a tailored distribution, Ordered-logit, that encapsulate all the above

$$\begin{aligned}
 R_i &\sim \text{Ordered-logit}(\varphi_i, \kappa_k) \\
 \varphi_i &= \text{terms of the linear model} \\
 \kappa_k &\sim \text{Normal}(0, 1.5)
 \end{aligned}$$

In [33]: `np.arange(6) - 2.5`

Out[33]: `array([-2.5, -1.5, -0.5, 0.5, 1.5, 2.5])`

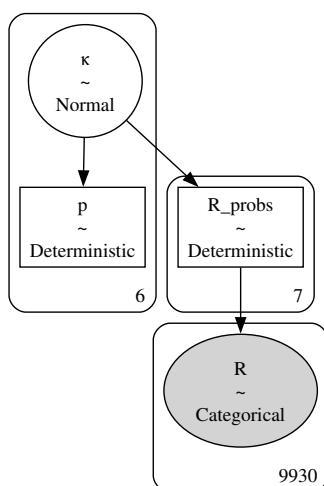
```
In [34]: with pm.Model() as m12_4:
          κ = pm.Normal('κ', mu=0, sigma=1.5,
                        transform=pm.distributions.transforms.ordered, # tells pymc that the vector of parameters must be ordered
                        shape=6,
                        initval=np.arange(6) - 2.5) # ensure that the initial values are ordered

          φ = 0
          R = pm.OrderedLogistic('R', φ, κ, observed=d_tr.response.values-1) # 0 lowest, 6 maximum

          # cumulative probabilities in cumulative scale
          p = pm.Deterministic('p', pm.math.invlogit(κ))
```

In [35]: `pm.model_to_graphviz(m12_4)`

Out[35]:



In []: `trace_m12_4 = pm.sample(model=m12_4)`

In [37]: `pm.summary(trace_m12_4, round_to=2)`

```
Out[37]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
$\kappa[0]$	-1.92	0.03	-1.97	-1.86	0.0	0.0	4094.91	3120.35	1.0
$\kappa[1]$	-1.27	0.02	-1.31	-1.22	0.0	0.0	4953.55	3571.41	1.0
$\kappa[2]$	-0.72	0.02	-0.76	-0.68	0.0	0.0	4683.33	3718.00	1.0
$\kappa[3]$	0.25	0.02	0.21	0.29	0.0	0.0	4913.97	3323.28	1.0
$\kappa[4]$	0.89	0.02	0.85	0.93	0.0	0.0	5213.74	3364.79	1.0
$\kappa[5]$	1.77	0.03	1.71	1.82	0.0	0.0	5381.44	3794.96	1.0
R_probs[0]	0.13	0.00	0.12	0.13	0.0	0.0	4094.91	3120.35	1.0
R_probs[1]	0.09	0.00	0.09	0.10	0.0	0.0	5319.78	3128.27	1.0
R_probs[2]	0.11	0.00	0.10	0.11	0.0	0.0	5495.95	3235.85	1.0
R_probs[3]	0.23	0.00	0.23	0.24	0.0	0.0	4995.64	3006.79	1.0
R_probs[4]	0.15	0.00	0.14	0.15	0.0	0.0	5852.17	3177.46	1.0
R_probs[5]	0.15	0.00	0.14	0.15	0.0	0.0	5403.26	3038.74	1.0
R_probs[6]	0.15	0.00	0.14	0.15	0.0	0.0	5381.44	3794.96	1.0
p[0]	0.13	0.00	0.12	0.13	0.0	0.0	4094.91	3120.35	1.0
p[1]	0.22	0.00	0.21	0.23	0.0	0.0	4953.55	3571.41	1.0
p[2]	0.33	0.00	0.32	0.34	0.0	0.0	4683.33	3718.00	1.0
p[3]	0.56	0.00	0.55	0.57	0.0	0.0	4913.97	3323.28	1.0
p[4]	0.71	0.00	0.70	0.72	0.0	0.0	5213.74	3364.79	1.0
p[5]	0.85	0.00	0.85	0.86	0.0	0.0	5381.44	3794.96	1.0

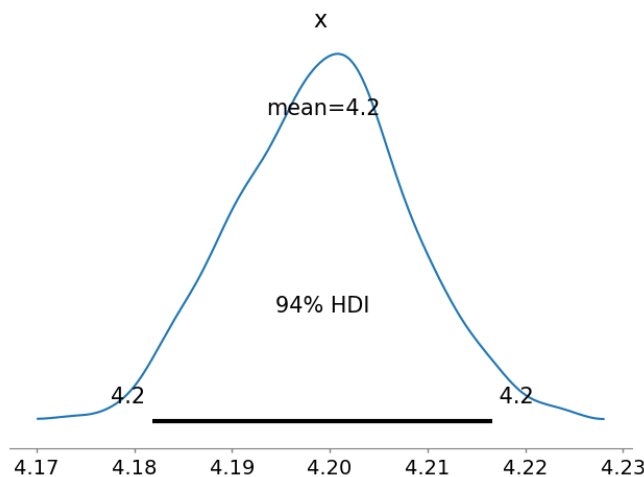
```
In [38]: trace_m12_4.posterior.R_probs.mean(['chain', 'draw']).values
```

```
Out[38]: array([0.1282847, 0.09156094, 0.10785844, 0.23398921, 0.14715303,
0.14547396, 0.14567972])
```

```
In [39]: # expected outcome
Ep = np.dot(trace_m12_4.posterior.R_probs.mean(['chain', 'draw']).values, np.arange(1,8))
Ep
```

```
Out[39]: 4.1993056984919
```

```
In [40]: pm.plot_posterior(np.array([np.dot(x, np.arange(1,8)) for x in trace_m12_4.posterior.R_probs.mean(['chain']).values]));
```



Adding predictor variables

- The model above is simply a Bayesian version of a histogram
- We can easily add predictors in the φ_i parameter
- Here we will use a linear model

$$\log \frac{\Pr(y_i \leq k)}{1 - \Pr(y_i \leq k)} = \alpha_k - \varphi_i$$

$$\varphi_i = \beta x_i$$

- Why minus (-) above? Because deduction values preserves the ordered scale
 - The expected outcome increases if we decrease the log-odds for all outcomes
 - Recall: $\mathbb{E}[y] = \sum_i p_i y_i$ where y_i represent the value of outcome i and p_i is the probability the outcome.
 - The code below demonstrates this effect

```
In [41]: Ep
```

```
Out[41]: 4.1993056984919
```

```
In [42]: cumprobs2 = logistic(trace_m12_4.posterior.k.mean(['chain', 'draw']).values+0.5)
probs2 = [cumprobs2[0]] + [cumprobs2[i]-cumprobs2[i-1] for i in range(1,6)] + [1-cumprobs2[5]]
np.dot(probs2, np.arange(1,8))
```

```
Out[42]: 3.6564317168984624
```

Adding action, intention and contact

- The trolley dataset contains 3 indicator variables we will use as predictors
 - action - The actor perform an action to create the outcome (e.g., pulls the lever)

- intention - The actor behaves depending on the outcome she wants to achieve
- contact - The actor uses physical contact to cause harm to a victim
- The predictors define 3 versions of the riddle aiming at testing their impact
- The model should consider the following possibilities:
 1. No action, contact, or intention
 2. Action
 3. Contact
 4. Intention
 5. Action and Intention
 6. Contact and intention
- To this end the cumulative-log-odds is defined as:

$$\log \frac{\Pr(y_i \leq k)}{1 - \Pr(y_i \leq k)} = \alpha_k - \varphi_i$$

$$\varphi_i = \beta_A A_i + \beta_C C_i + B_{I,i} I_i$$

$$B_{I,i} = \beta_I + \beta_{IA} A_i + \beta_{IC} C_i$$

- [QUESTION]: Is this an interactive combination of predictors?
- [QUESTION]: Could we map the different combinations into a treatment variable? Would that be reasonable?

```
In [43]: with pm.Model() as m12_5:
    alpha = pm.Normal('alpha', mu=0, sigma=1.5,
                      transform=pm.distributions.transforms.ordered, # tells pymc3 that this parameters are log-cumulative-odds of an ordinal scale
                      shape=6,
                      initval=np.arange(6) - 2.5) # ensure that the initial values are ordered

    beta_A = pm.Normal('beta_A', mu=0, sigma=0.5)
    beta_I = pm.Normal('beta_I', mu=0, sigma=0.5)
    beta_C = pm.Normal('beta_C', mu=0, sigma=0.5)
    beta_IA = pm.Normal('beta_IA', mu=0, sigma=0.5)
    beta_IC = pm.Normal('beta_IC', mu=0, sigma=0.5)

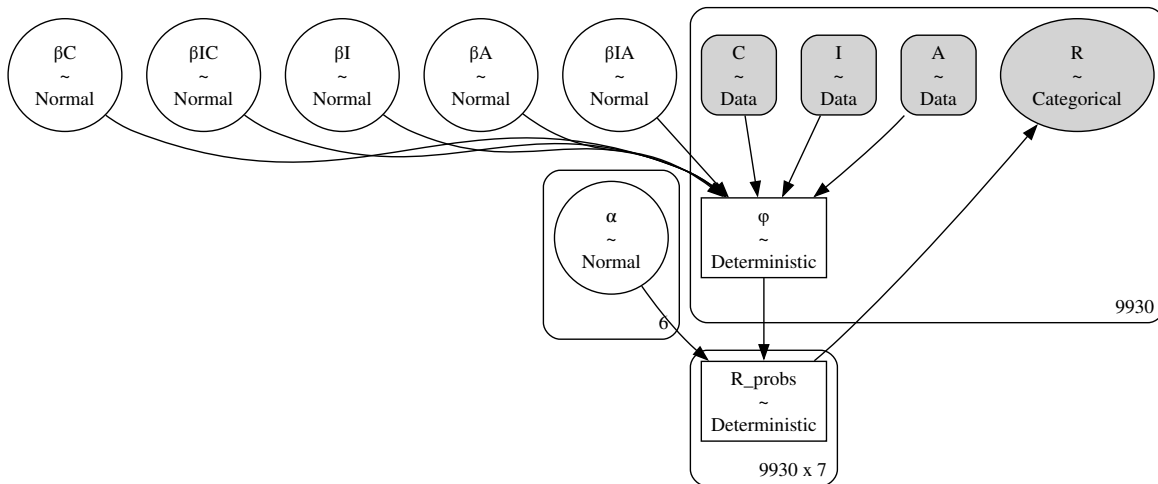
    # Predictors from data
    A = pm.Data("A", d_tr.action.values)
    I = pm.Data("I", d_tr.intention.values)
    C = pm.Data("C", d_tr.contact.values)

    BI = beta_I + beta_IA*A + beta_IC*C
    phi = pm.Deterministic('phi', beta_A*A + beta_C*C + BI*I)

    R = pm.OrderedLogistic('R', phi, alpha, observed=d_tr.response.values-1) # 0 lowest, 6 maximum
```

In [44]: pm.model_to_graphviz(m12_5)

Out[44]:

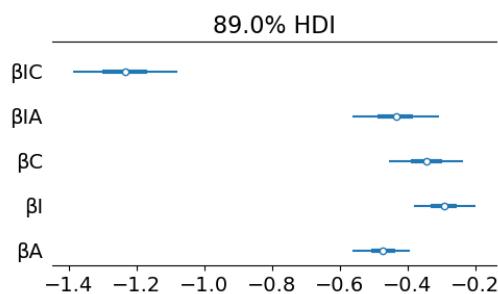


In []: trace_m12_5 = pm.sample(model=m12_5)

In [46]: pm.summary(trace_m12_5, var_names=['beta_IC', 'beta_IA', 'beta_C', 'beta_I', 'beta_A'])

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
beta_IC	-1.235	0.097	-1.412	-1.050	0.002	0.001	2889.0	2732.0	1.0
beta_IA	-0.434	0.080	-0.578	-0.281	0.002	0.001	2667.0	2887.0	1.0
beta_C	-0.343	0.069	-0.466	-0.211	0.001	0.001	2882.0	2764.0	1.0
beta_I	-0.291	0.057	-0.398	-0.182	0.001	0.001	2328.0	2638.0	1.0
beta_A	-0.472	0.053	-0.576	-0.375	0.001	0.001	2502.0	2949.0	1.0

In [47]: pm.plot_forest(trace_m12_5, var_names=['beta_IC', 'beta_IA', 'beta_C', 'beta_I', 'beta_A'], combined=True, hdi_prob=.89, figsize=(5, 3));



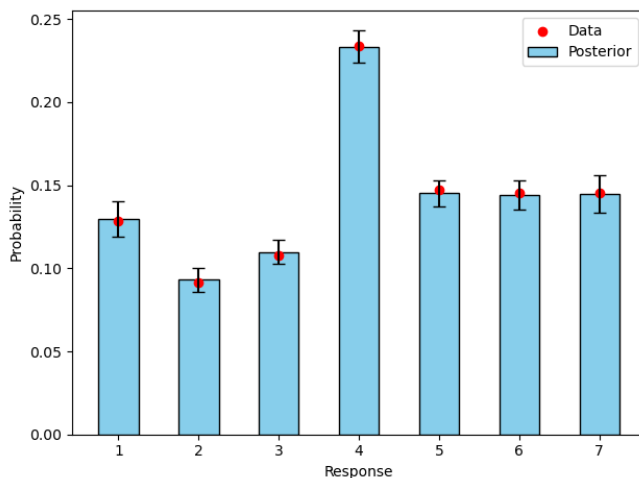
Posterior predictive check and posterior predictions

- It is somehow difficult to effectively plot the posterior predictive check or posterior predictions for ordinal models
 - The reason is that, each sample, is a vector of probabilities that depends on the values of the predictors
- To generate posterior predictions, a standard approach is to pick a predictor and plot the cumulative probability of all outcomes for all values of the predictor
 - This requires fixing the values of the other predictors if any
- Below we show two possibilities (a bit different from what it is shown in the book) for a posterior predictive check (1) and to generate posterior predictions (2)
 - Posterior predictive check.** We compare the posterior probability of each outcome (with the 95% HDI)—this is essentially a Bayesian version of a histogram, and the frequency of each answer in the data
 - Generating posterior predictions.** We use different predictor values and comparing the different in response probabilities
 - We consider fix different values of `action` and `contact` and compare the effect of `intention`

```
In [48]: # data probability
prct_dict = d_tr.response.value_counts(normalize=True, ascending=True).to_dict()
sorted_prct = [prct_dict[k] for k in sorted(prct_dict)]

# posterior probability (mean and hdi)
means = trace_m12_5.posterior.R_probs.mean(dim=['chain','draw','R_probs_dim_0'])
hdis = pm.hdi(trace_m12_5.posterior.R_probs, hdi_prob=.95).mean(dim=['R_probs_dim_0']).R_probs

# Posterior predictive check using posterior distribution of probability
categories = np.arange(1,8)
plt.bar(categories, means, yerr=[means-hdis[:,0], hdis[:,1]-means],
        capsize=4, color='skyblue', edgecolor='black', width=0.5, label='Posterior')
plt.scatter(categories, sorted_prct, color='red', marker='o', label='Data')
plt.xlabel('Response')
plt.ylabel('Probability')
plt.legend()
plt.show();
```



```
In [49]: # Posterior predictive analysis for probability (a bit different than McElreath's)
fig, axs = plt.subplots(1,3, figsize=(15,4))

for ((a,c),ax) in zip([(0,0), (1,0), (0,1)],axs):

    # posterior predictions
    with m12_5:
        ## no action, no contact, no interaction
        pm.set_data({'A': [a], 'C': [c], 'I': [0]})
        trace_m12_5_pp_0 = pm.sample_posterior_predictive(trace_m12_5, var_names=['R_probs'])
        ## no action, no contact, interaction
        pm.set_data({'A': [a], 'C': [c], 'I': [1]})
        trace_m12_5_pp_1 = pm.sample_posterior_predictive(trace_m12_5, var_names=['R_probs'])

    categories = np.arange(1,8)
    values1 = trace_m12_5_pp_0.posterior_predictive.R_probs.mean(dim=['chain','draw']).sel(R_probs_dim_2=0).values
    values2 = trace_m12_5_pp_1.posterior_predictive.R_probs.mean(dim=['chain','draw']).sel(R_probs_dim_2=0).values
    errors1 = [values1 - pm.hdi(trace_m12_5_pp_0.posterior_predictive.R_probs).R_probs.values[0,:,0],
               pm.hdi(trace_m12_5_pp_0.posterior_predictive.R_probs).R_probs.values[0,:,1]-values1]
    errors2 = [values2 - pm.hdi(trace_m12_5_pp_1.posterior_predictive.R_probs).R_probs.values[0,:,0],
               pm.hdi(trace_m12_5_pp_1.posterior_predictive.R_probs).R_probs.values[0,:,1]-values2]

    # Number of categories
    n = len(categories)

    # Positions of the bars on the x-axis
    ind = np.arange(n)

    # Width of the bars
    width = 0.35

    # Create bar plots
    bar1 = ax.bar(ind - width/2, values1, width, yerr=errors1,
                  capsize=4, label='Intention: 0', color='gray', edgecolor='black')
    bar2 = ax.bar(ind + width/2, values2, width, yerr=errors2,
                  capsize=4, label='Intention: 1', color='cyan', edgecolor='black')

    ax.set_xlabel('Response')
    ax.set_ylabel('Probability')
    ax.set_title(f'Action: {a}, Contact: {c}')
    ax.set_xticks(ind)
    ax.set_xticklabels(categories)
    ax.legend()

# Show plot
plt.show()
```

Sampling: []
Output()

Sampling: []

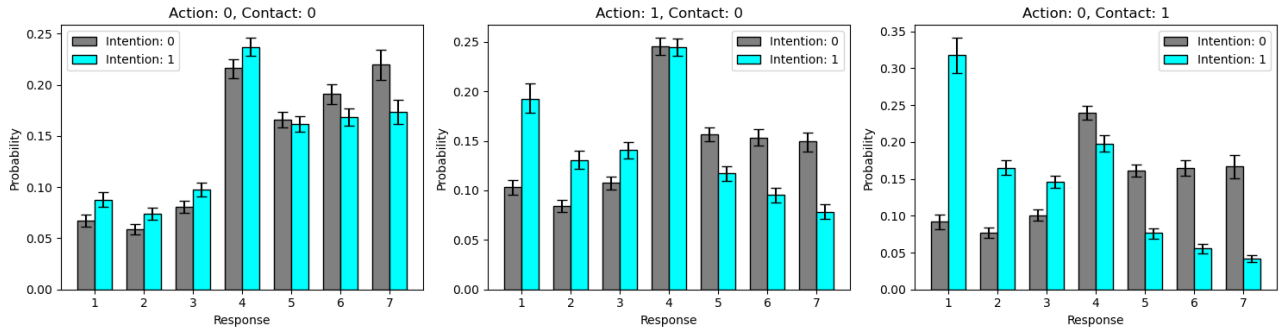
```
Output()

Sampling: []
Output()

Sampling: []
Output()

Sampling: []
Output()

Sampling: []
Output()
```



Ordered Predictor

- We add a predictor in the ordinal scale
- **[QUESTION]: What type of predictors did we consider in the model?**
- We use education, `edu` in the `Trolley` dataset, as an ordinal predictor.

```
In [50]: d_tr
```

```
Out[50]:
```

	case	response	order	id	age	male	edu	action	intention	contact	story	action2
0	cfaqu	4	2	96;434	14	0	Middle School	0	0	1	aqu	1
1	cfbur	3	31	96;434	14	0	Middle School	0	0	1	bur	1
2	cfrub	4	16	96;434	14	0	Middle School	0	0	1	rub	1
3	cibox	3	32	96;434	14	0	Middle School	0	1	1	box	1
4	cibur	3	4	96;434	14	0	Middle School	0	1	1	bur	1
...	...	...	...	...	...	...	...	...	...	...	...	...
9925	ilpon	3	23	98;299	66	1	Graduate Degree	0	1	0	pon	0
9926	ilsha	6	15	98;299	66	1	Graduate Degree	0	1	0	sha	0
9927	ilshi	7	7	98;299	66	1	Graduate Degree	0	1	0	shi	0
9928	ilswi	2	18	98;299	66	1	Graduate Degree	0	1	0	swi	0
9929	nfrub	2	17	98;299	66	1	Graduate Degree	1	0	0	rub	1

9930 rows × 12 columns

```
In [51]: d_tr["edu_new"] = pd.Categorical(
    d_tr.edu.values,
    categories=[
        "Elementary School", # lowest
        "Middle School",
        "Some High School",
        "High School Graduate",
        "Some College",
        "Bachelor's Degree",
        "Master's Degree",
        "Graduate Degree", # highest
    ],
    ordered=True,
)
d_tr["edu_new"] = d_tr.edu_new.cat.codes
```

- As for outcomes, the notion of ordered predictor implies an increment as you ascend in the ordinal scale
- We will use a parameter δ_i for each element in the ordinal scale
- To encode the cumulative nature of the scale we add in a linear fashion

$$\begin{array}{lll}
 \text{(Individual at level 1)} & \varphi_i = \delta_1 & + \text{other predictors in the linear model} \\
 \text{(Individual at level 2)} & \varphi_i = \delta_1 + \delta_2 & + \text{other predictors in the linear model} \\
 \dots & \dots & \dots \\
 \text{(Individual at level 7)} & \varphi_i = \delta_1 + \delta_2 + \dots + \delta_7 & + \text{other predictors in the linear model}
 \end{array}$$

- The above can be compactly expressed as

$$\varphi_i = \sum_{j=0}^{E_i-1} \delta_j + \text{other predictors in the linear model}$$

where E_i denotes the education level of the i th individual.

- Note that we have introduced a term δ_0 , this is a technicality to be able to write the model in a compact form. In fact, it is a constant $\delta_0 = 0$ as the parameter for the lowest ordinal value is the intercept.
- To complete the term, we multiply it by a coefficient β_E which captures the effect of education on the outcome
- The model above is defined as follows to include the ordinal predictor:

$$\begin{aligned}
 R_i &\sim \text{Ordered} - \text{logit}(\varphi_i, \kappa_k) \\
 \varphi_i &= \underbrace{\beta_E \sum_{j=0}^{E_i-1} \delta_j}_{\text{Ordinal predictor}} + \beta_A A_i + \beta_C C_i + B_{I,i} I_i \\
 B_{I,i} &= \beta_I + \beta_{IA} A_i + \beta_{IC} C_i \\
 \kappa_k &\sim \text{Normal}(0, 1.5) \\
 \beta_A, \beta_I, \beta_C, \beta_E &\sim \text{Normal}(0, 1) \\
 \delta &\sim \text{Dirichlet}(\alpha)
 \end{aligned}$$

Dirichlet distribution

- It is an instance of a *multivariate* distribution
 - Probability distribution involving more than one variable
- It is a generalization of Beta
- Takes as input a vector α of pseudo-counts for each variable
 - Each element of the vector indicates how often the variable in that position appears in comparison with the others
- The support is a vector of real numbers in $(0, 1)$ that sum up to 1, i.e., a *simplex* vector.
 - This property is what makes it appropriate as a prior for δ**
- See the bonus material on the Dirichlet distribution (directory 110-mixtures/dirichlet-bonus-material/) for additional examples and an exercise

```
In [52]: with pm.Model() as m12_6:
         alpha = pm.Normal('alpha', mu=0, sigma=1.5,
                           transform=pm.distributions.transforms.ordered, # tells pymc that this parameters are log-cumulative-odds of an ordinal scale
                           shape=6,
                           initial=np.arange(6) - 2.5) # ensure that the initial values are ordered
         beta_A = pm.Normal('beta_A', mu=0, sigma=0.5)
         beta_I = pm.Normal('beta_I', mu=0, sigma=0.5)
         beta_C = pm.Normal('beta_C', mu=0, sigma=0.5)
         beta_IA = pm.Normal('beta_IA', mu=0, sigma=0.5)
         beta_IC = pm.Normal('beta_IC', mu=0, sigma=0.5)
         beta_E = pm.Normal('beta_E', mu=0, sigma=0.5)

         delta = pm.Dirichlet('delta', np.repeat(2.0, 7), shape=7)
         delta_i = tt.concatenate([tt.zeros(1), delta])

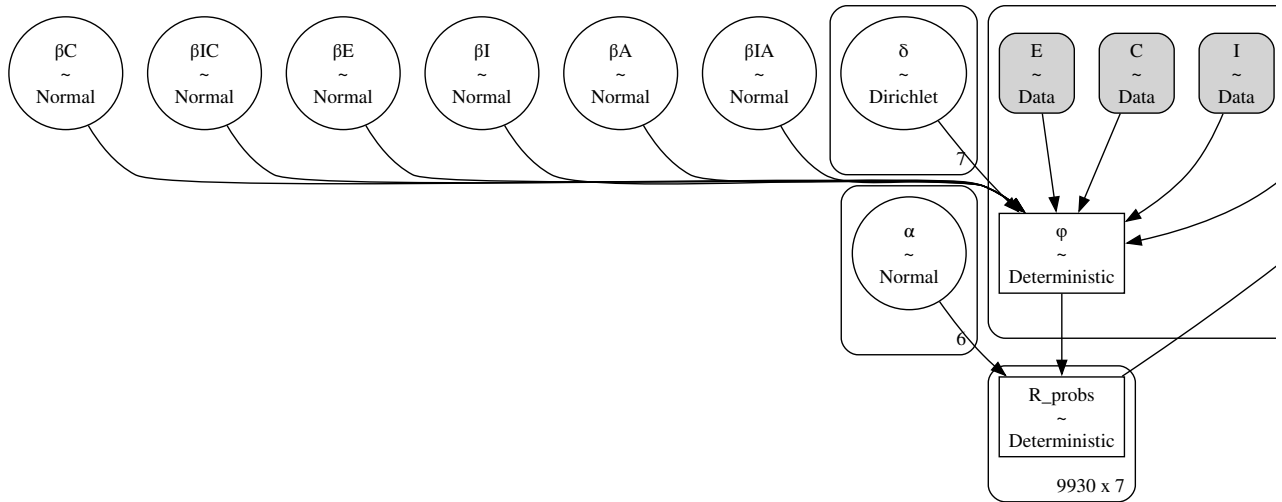
         # Predictors from data
         A = pm.Data("A", d_tr.action.values)
         I = pm.Data("I", d_tr.intention.values)
         C = pm.Data("C", d_tr.contact.values)
         E = pm.Data("E", d_tr.edu_new.values)

         BI = beta_I + beta_IA*A + beta_IC*C
         phi = pm.Deterministic('phi', beta_E*(tt.cumsum(delta_i)[E]) + beta_A*A + beta_C*C + BI*I)

         R = pm.OrderedLogistic('R', phi, alpha, observed=d_tr.response.values-1) # 0 lowest, 6 maximum
```

```
In [53]: pm.model_to_graphviz(m12_6)
```

```
Out[53]:
```



```
In [ ]: trace_m12_6 = pm.sample(model=m12_6, target_accept=.9)
```

```
In [55]: pm.summary(trace_m12_6, var_names=['beta_IC', 'beta_IA', 'beta_C', 'beta_I', 'beta_A', 'beta_E', 'delta'], round_to=2)
```

Out[55]:	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
β_{IC}	-1.24	0.10	-1.43	-1.07	0.0	0.0	2119.12	2473.17	1.0
β_{IA}	-0.44	0.08	-0.58	-0.28	0.0	0.0	2204.99	2197.33	1.0
β_C	-0.34	0.07	-0.46	-0.21	0.0	0.0	2481.95	2734.69	1.0
β_I	-0.29	0.06	-0.39	-0.19	0.0	0.0	2034.33	2548.27	1.0
β_A	-0.47	0.05	-0.57	-0.36	0.0	0.0	2411.99	2329.11	1.0
β_E	-0.32	0.15	-0.63	-0.04	0.0	0.0	1475.74	1243.62	1.0
$\delta[0]$	0.22	0.13	0.01	0.46	0.0	0.0	2017.53	2330.05	1.0
$\delta[1]$	0.14	0.09	0.01	0.30	0.0	0.0	3587.61	2390.30	1.0
$\delta[2]$	0.19	0.11	0.01	0.38	0.0	0.0	2982.56	1749.57	1.0
$\delta[3]$	0.18	0.10	0.02	0.36	0.0	0.0	3107.63	2522.03	1.0
$\delta[4]$	0.04	0.05	0.00	0.10	0.0	0.0	1835.54	1847.93	1.0
$\delta[5]$	0.10	0.07	0.01	0.22	0.0	0.0	3271.45	2223.62	1.0
$\delta[6]$	0.13	0.08	0.01	0.26	0.0	0.0	4192.39	2497.33	1.0

- These results indicate Education has a *negative impact* on the outcome (morality level)
 - The more educate the lower morality score the participants selects

```
In [56]: trace_m12_6.posterior
```

```
Out[56]: xarray.Dataset
```

► Dimensions:	(chain: 4, draw: 1000, α_{dim_0} : 6, δ_{dim_0} : 7, ϕ_{dim_0} : 9930, $R_{probs_dim_0}$: 9930, $R_{probs_dim_1}$: 7)			
▼ Coordinates:				
chain	(chain)	int64	0 1 2 3	
draw	(draw)	int64	0 1 2 3 4 5 ... 995 99...	
α_{dim_0}	(α_{dim_0})	int64	0 1 2 3 4 5	
δ_{dim_0}	(δ_{dim_0})	int64	0 1 2 3 4 5 6	
ϕ_{dim_0}	(ϕ_{dim_0})	int64	0 1 2 3 4 ... 9926 992...	
$R_{probs_dim_0}$	($R_{probs_dim_0}$)	int64	0 1 2 3 4 ... 9926 992...	
$R_{probs_dim_1}$	($R_{probs_dim_1}$)	int64	0 1 2 3 4 5 6	
▼ Data variables:				
β_A	(chain, draw)	float64	-0.4961 -0.4346 ... -0....	
β_I	(chain, draw)	float64	-0.3512 -0.184 ... -0.3...	
β_C	(chain, draw)	float64	-0.3369 -0.3401 ... -0....	
β_{IA}	(chain, draw)	float64	-0.3451 -0.5241 ... -0....	
β_{IC}	(chain, draw)	float64	-1.185 -1.335 ... -1.20...	
β_E	(chain, draw)	float64	0.02204 -0.5701 ... -0...	
α	(chain, draw, α_{dim_0})	float64	-2.62 -1.908 ... 0.149...	
δ	(chain, draw, δ_{dim_0})	float64	0.0981 0.1757 ... 0.0...	
ϕ	(chain, draw, ϕ_{dim_0})	float64	-0.3347 -0.3347 ... -0....	
R_{probs}	(chain, draw, $R_{probs_dim_0}$, $R_{probs_dim_1}$)	float64	0.09233 0.07945 ... 0...	
► Indexes: (7)				
▼ Attributes:				
created_at:	2025-04-09T08:50:58.024963+00:00			
arviz_version:	0.20.0			
inference_library:	pymc			
inference_libra...	5.19.1			
sampling_time:	269.39456129074097			
tuning_steps:	1000			

```
In [57]: trace_m12_6.posterior.coords[" $\delta_{dim\_0}$ "] = [
    "Elem",
    "MidSch",
    "SHS",
    "HSG",
    "SCoL",
    "Bach",
    "Mast",
]

pm.plot_pair(
    trace_m12_6,
    var_names=[" $\delta$ "],
    marginals=True,
    point_estimate="mean"
);
```

